ECE 220 Computer Systems & Programming

Lecture 18 — Problem Solving with Linked Lists

October 27, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

Why &head?

```
int main() {
    /* a one-node list, built in main */
    Node *head;
    head = (Node *)malloc(sizeof(Node));
    if (head == NULL) return 1;
    head->UIN = 246813579;              /* Robin */
    head->GPA = 3.52;
    head->next = NULL;

    /* add a node by calling another function */
    add_node(&head, 123456789, 3.89);   /* Jamie */
    print_list(head);

    free_list(head);
    return 0;
}
```

`add_node`, `print_list` and `free_list` are the recursive functions of `llist.c`.

Why &head?

```c
int main() {
    /* a one-node list, built in main */
    Node *head;
    head = (Node *)malloc(sizeof(Node));
    if (head == NULL) return 1;
    head->UIN = 246813579;              /* Robin */
    head->GPA = 3.52;
    head->next = NULL;

    /* add a node by calling another function */
    add_node(&head, 123456789, 3.89);   /* Jamie */
    print_list(head);

    free_list(head);
    return 0;
}
```

`add_node`, `print_list` and `free_list` are the recursive functions of `llist.c`.

It prints

```
123456789 3.89
246813579 3.52
```

Jamie's UIN is below Robin's, so Jamie's node goes in front, and `main`'s `head` now points at it. `add_node` changed a variable that lives in `main`.

By the end of today you will know why `main` passes `&head` and what `add_node` does with it, and you will build a stack and a queue from the same kind of node.

Today's Three Questions

`add_node(&head, 123456789, 3.89);`
changes `head`. `add_node(head, ...)` cannot.

1. Where do `head`, `add_node`'s parameter `list`, and the nodes live during the call, and what does each hold?
2. What does a second pointer in each node, `prev`, buy, and what does it cost?
3. How does a stack, and then a queue, sit on a linked list, and at which end does each operation work?

Answers today: `list` holds the address of `head`; a node can be reached and unlinked from either side, for 8 more bytes; a stack works at the head only, a queue adds at the tail and removes at the head.

Pass `head` Itself: `byvalue.c`

```
void add_node(Node *list,
              int new_UIN, float new_GPA) {
    Node *new_node;
    /* base case 1: insert new node at 'head' */
    if (list == NULL || list->UIN > new_UIN) {
        new_node = (Node *)malloc(sizeof(Node));
        if (new_node == NULL) return;
        new_node->UIN = new_UIN;
        new_node->GPA = new_GPA;
        new_node->next = list;
        list = new_node;              /* the copy */
        return;
    }
    /* base case 2: UIN already exists */
    if (list->UIN == new_UIN) return;
    /* recursive case: the remaining list */
    add_node(list->next, new_UIN, new_GPA);
}
```

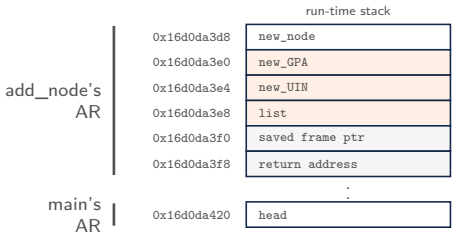
`main` is unchanged except for the call:
`add_node(head, 123456789, 3.89);`

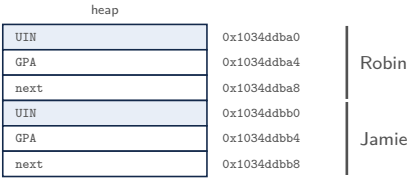
- It prints _____
- `list` starts as a copy of _____
- `list = new_node;` changes _____; `main`'s `head` still points at _____
- Jamie's node is _____
- `leaks --atExit -- ./byvalue`: _____

To change `main`'s `head`, `add_node` needs the address of `head`. The RTS during the call, next.

`add_node(&head, 123456789, 3.89)` on the RTS

Inside `add_node`, just after `*list = new_node`. Fill in the cells and draw each pointer as an arrow.





- `list` holds _____, so `*list` is _____
- Before `*list = new_node`, `head` held _____

- `add_node(&head, 135792468, 3.71)` adds Casey next. Its second call's `list` holds 0x1034ddbb8, the address of _____

As with `swap(&x, &y)`: to change `main`'s `head`, pass `&head`.

Addresses from one run of `where.c` on a Mac; lower addresses at the top.

When Do We Need a Double Pointer?

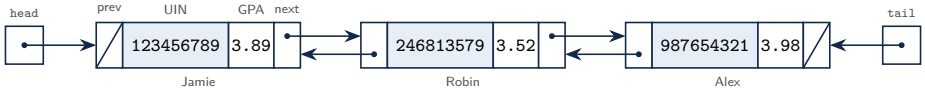
A function needs the address of a pointer when it must change the pointer that the caller holds.

Function	Changes a pointer the caller holds?	Parameter
`print_list(head)`	_____	_____
`find_node(head, S_UIN)`	_____	_____
`add_node(&head, ...)`	_____	_____
`remove_node(&head, ...)`	_____	_____
`free_list(head)`	_____	_____

- After `free_list(head)`, `head` still holds _____; reading through it is an error

Doubly Linked List

Each node also holds the address of the node before it.

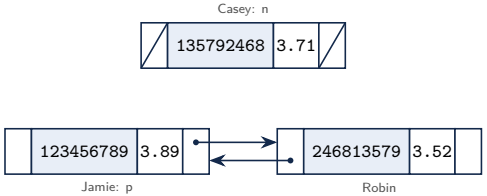



```
typedef struct studentStruct Node;
struct studentStruct {
    int   UIN;
    float GPA;
    Node *prev;
    Node *next;
};
```

- The first node's `prev` and the last node's `next` are _____
- `sizeof(Node)` on the laptop: _____
- From `tail`, the walk `p = p->prev` prints the list _____
- To unlink a node `p`, a singly linked list needs _____, found by a walk from `head`; here it is _____
- Each insert or remove sets twice as many pointers

In-Class Problem: `insert_after`

Insert Casey, `n`, after Jamie, `p`. Draw the pointers after the call.



- Pointers set: singly linked ___, doubly linked ___
- `p->next = n` comes last because _____
- Insert before Jamie: _____

```
/* insert node n after node p (p not NULL) */
void insert_after(Node *p, Node *n) {
    /* n points back at p, forward at p's next */



    /* the node after p, if any, points back at n */



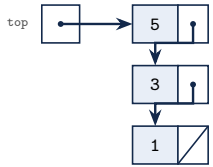
    /* p points forward at n */


}
```

The Stack ADT on a Linked List

- The first item in is the last item out: _____. Two operations: _____ and _____
- An abstract data type (ADT) is defined by its operations. The LC-3 stack held words in a fixed block of memory: a push on a full stack is _____, a pop on an empty one _____
- Here each push `malloc`s one node, so the stack is full only when _____. Empty: `top ==` _____
- `top` is the list's _____. Push and pop each set ___ pointers and walk _____
- Pop at the other end would need _____: a walk of the whole list

After `push(1)`, `push(3)`, `push(5)`: draw `push(7)`, then `pop()`.



`pop()` returns ____; `top` is back at ____

`top` is a **global** in `stack.c`, so `push` and `pop` change it with no pointer parameter at all — where `add_node` needed `&head`.

In-Class Problem: `push` and `pop`

A `node` holds `int data` and `node *next`. `top` is a global variable, so neither function takes the stack as a parameter. `pop` returns -1 on underflow.

```
/* push a new node onto the stack */
void push(int new_data) {
    node *new_node = (node *)malloc(sizeof(node));
    /* overflow: malloc returned NULL */



    /* fill new_node; link it above the old top */



}
```

```
/* pop the top node; return its data (-1: empty) */
int pop() {
    node *old;
    int value;
    /* underflow: the stack is empty */



    /* unlink and free the top; return its data */

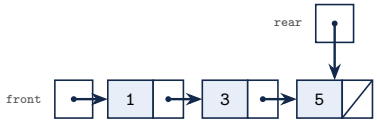


}
```

The Queue ADT on a Linked List

- The first item in is the first item out: _____. Two operations: _____ and _____
- `front` is the list's head; `rear` points at the _____. Both are globals, like `top`: `enqueue` and `dequeue` take no pointer parameter either
- Enqueue at the _____: `rear->next = new_node`, no walk. Dequeue at the _____: the same unlink as `pop`
- The other way round, dequeue at the rear would need _____
- Empty: `front == NULL` and `rear == _____`. The last dequeue must also set _____

After `enqueue(1)`, `enqueue(3)`, `enqueue(5)`: draw `enqueue(7)`, then `dequeue()`.



`dequeue()` returns ____; `front` moves to ____

A **deque** (double-ended queue) adds and removes at both ends. On a doubly linked list with `front` and `rear`, removing at the rear is `rear = rear->prev`: all four operations run without a walk.

The keyboard input buffer is a queue in a circular array.

In-Class Problem: enqueue and dequeue

```
void enqueue(int new_data) {
    node *new_node = (node *)malloc(sizeof(node));
    if (new_node == NULL) return;   /* full */
    /* fill new_node: it will be the last node */



    /* link it after the rear node, or as front */



    /* it is the new rear */


}
```

```
int dequeue() {
    node *old;
    int value;
    if (front == NULL) return -1;   /* empty */
    /* unlink the front node */



    /* the queue may now be empty: fix rear */


    /* free the node; return its data */


}
```

Summary & What's Next

Today

- **The problem**: `add_node(&head, ...)` changes `main`'s `head`; `add_node(head, ...)` changes a copy, and the new node leaks
- **The RTS**: `list` in `add_node`'s AR holds the address of `head` in `main`'s AR, or of a `next` field on the heap; `*list = new_node` writes there. A function that changes a caller's pointer takes `Node **`
- **Doubly linked**: `prev` and `next`; walk both ways, unlink without a walk; 24 bytes per node, four pointers per insert
- **Stack**: LIFO; `top` is the head; push and pop at the head with no walk; full only when `malloc` fails
- **Queue**: FIFO; enqueue at `rear`, dequeue at `front`; reset `rear` when the queue empties. A deque on a doubly linked list works at both ends

Next lecture

- From C to LC-3 with linked data: `print_list` converted to LC-3 by hand, a node as two words in memory, and the RTS of the recursive calls

Code from today: `addhead.c`, `byvalue.c`, `where.c`, `dlist.c`, `stack.c`, `queue.c` (skeletons `dlist_skel.c`, `stack_skel.c`, `queue_skel.c`).